

Hitters Notebook

[Code ▼](#)

The hitters dataset consists of 322 observations of 21 variables with the following information - X, AtBat, Hits, HmRun (home runs), Runs, RBI, Walks, Years, CAtBat, CHits, CHmRun, CRuns, CRBI, CWalks, League, Division, PutOuts, Assists, Errors, Salary, New League. The variable definitions are:

X: Name

AtBat: Number of times at bat in 1986

Hits: Number of hits in 1986

HmRun: Number of home runs in 1986

Runs: Number of runs in 1986

RBI: Number of runs batted in in 1986

Walks: Number of walks in 1986

Years: Number of years in the major leagues

CAtBat: Number of times at bat during his career

CHits: Number of hits during his career

CHmRun: Number of home runs during his career

CRuns: Number of runs during his career

CRBI: Number of runs batted in during his career

CWalks: Number of walks during his career

League: A and N indicating player's league at the end of 1986

Division: E and W indicating player's division at the end of 1986

PutOuts: Number of put outs in 1986

Assists: Number of assists in 1986

Errors: Number of errors in 1986

Salary: 1987 annual salary on opening day in thousands of dollars

NewLeague: A and N indicating player's league at the beginning of 1987.

Here League, Division and NewLeagues are variables with 2 categories. We drop rows with missing entries and are left with 263 observations.

[Hide](#)

```
rm(list=ls())
hitters <- read.csv("hitters.csv")
str(hitters)
```

```
'data.frame': 322 obs. of 21 variables:
 $ X      : chr  "-Andy Allanson" "-Alan Ashby" "-Alvin Davis" "-Andre Dawson" ...
 $ AtBat   : int   293 315 479 496 321 594 185 298 323 401 ...
 $ Hits    : int   66 81 130 141 87 169 37 73 81 92 ...
 $ HmRun   : int    1 7 18 20 10 4 1 0 6 17 ...
 $ Runs    : int   30 24 66 65 39 74 23 24 26 49 ...
 $ RBI     : int   29 38 72 78 42 51 8 24 32 66 ...
 $ Walks   : int   14 39 76 37 30 35 21 7 8 65 ...
 $ Years   : int    1 14 3 11 2 11 2 3 2 13 ...
 $ CAtBat  : int  293 3449 1624 5628 396 4408 214 509 341 5206 ...
 $ CHits   : int  66 835 457 1575 101 1133 42 108 86 1332 ...
 $ CHmRun  : int    1 69 63 225 12 19 1 0 6 253 ...
 $ CRuns   : int   30 321 224 828 48 501 30 41 32 784 ...
 $ CRBI    : int   29 414 266 838 46 336 9 37 34 890 ...
 $ CWalks  : int   14 375 263 354 33 194 24 12 8 866 ...
 $ League  : chr   "A" "N" "A" "N" ...
 $ Division: chr   "E" "W" "W" "E" ...
 $ PutOuts : int  446 632 880 200 805 282 76 121 143 0 ...
 $ Assists : int   33 43 82 11 40 421 127 283 290 0 ...
 $ Errors  : int   20 10 14 3 4 25 7 9 19 0 ...
 $ Salary  : num  NA 475 480 500 91.5 750 70 100 75 1100 ...
 $ NewLeague: chr  "A" "N" "A" "N" ...
```

Hide

```
hitters<-na.omit(hitters)
str(hitters)
```

```
'data.frame': 263 obs. of 21 variables:
 $ X      : chr  "-Alan Ashby" "-Alvin Davis" "-Andre Dawson" "-Andres Galarraga" ...
 $ AtBat   : int   315 479 496 321 594 185 298 323 401 574 ...
 $ Hits    : int   81 130 141 87 169 37 73 81 92 159 ...
 $ HmRun   : int    7 18 20 10 4 1 0 6 17 21 ...
 $ Runs    : int   24 66 65 39 74 23 24 26 49 107 ...
 $ RBI     : int   38 72 78 42 51 8 24 32 66 75 ...
 $ Walks   : int   39 76 37 30 35 21 7 8 65 59 ...
 $ Years   : int   14 3 11 2 11 2 3 2 13 10 ...
 $ CAtBat  : int  3449 1624 5628 396 4408 214 509 341 5206 4631 ...
 $ CHits   : int  835 457 1575 101 1133 42 108 86 1332 1300 ...
 $ CHmRun  : int   69 63 225 12 19 1 0 6 253 90 ...
 $ CRuns   : int  321 224 828 48 501 30 41 32 784 702 ...
 $ CRBI    : int  414 266 838 46 336 9 37 34 890 504 ...
 $ CWalks  : int  375 263 354 33 194 24 12 8 866 488 ...
 $ League  : chr   "N" "A" "N" "N" ...
 $ Division: chr   "W" "W" "E" "E" ...
 $ PutOuts : int  632 880 200 805 282 76 121 143 0 238 ...
 $ Assists : int   43 82 11 40 421 127 283 290 0 445 ...
 $ Errors  : int   10 14 3 4 25 7 9 19 0 22 ...
 $ Salary  : num   475 480 500 91.5 750 ...
 $ NewLeague: chr  "N" "A" "N" "N" ...
 - attr(*, "na.action")= 'omit' Named int [1:59] 1 16 19 23 31 33 37 39 40 42 ...
 ..- attr(*, "names")= chr [1:59] "1" "16" "19" "23" ...
```

Subset selection

The leaps package in R does subset selection with the regsubsets function. By default, the maximum number of subsets, this function uses is 8. We extend this to do a complete subset selection by changing the default value of nvmax argument in this function. Note that CRBI is in the model with 1 to 6 variables but not in the model with 7 and 8 variables. In the display “*” indicates variable is included and “ ” indicates variable is not included.

[Hide](#)

```
#install.packages("leaps")
library(leaps)
?regsubsets
hitters <- hitters[,2:21]
model1 <- regsubsets(Salary~.,data=hitters)
summary(model1)
```

Subset selection object

Call: regsubsets.formula(Salary ~ ., data = hitters)

19 Variables (and intercept)

	Forced in	Forced out
AtBat	FALSE	FALSE
Hits	FALSE	FALSE
HmRun	FALSE	FALSE
Runs	FALSE	FALSE
RBI	FALSE	FALSE
Walks	FALSE	FALSE
Years	FALSE	FALSE
CAtBat	FALSE	FALSE
CHits	FALSE	FALSE
CHmRun	FALSE	FALSE
CRuns	FALSE	FALSE
CRBI	FALSE	FALSE
CWalks	FALSE	FALSE
LeagueN	FALSE	FALSE
DivisionW	FALSE	FALSE
PutOuts	FALSE	FALSE
Assists	FALSE	FALSE
Errors	FALSE	FALSE
NewLeagueN	FALSE	FALSE

1 subsets of each size up to 8

Selection Algorithm: exhaustive

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years
1 (1)	" "	" "	" "	" "	" "	" "	" "
2 (1)	" "	"*"	" "	" "	" "	" "	" "
3 (1)	" "	"*"	" "	" "	" "	" "	" "
4 (1)	" "	"*"	" "	" "	" "	" "	" "
5 (1)	"*"	"*"	" "	" "	" "	" "	" "
6 (1)	"*"	"*"	" "	" "	" "	"*"	" "
7 (1)	" "	"*"	" "	" "	" "	"*"	" "
8 (1)	"*"	"*"	" "	" "	" "	"*"	" "

	CAtBat	CHits	CHmRun	CRuns	CRBI	CWalks
1 (1)	" "	" "	" "	" "	"*"	" "
2 (1)	" "	" "	" "	" "	"*"	" "
3 (1)	" "	" "	" "	" "	"*"	" "
4 (1)	" "	" "	" "	" "	"*"	" "
5 (1)	" "	" "	" "	" "	"*"	" "
6 (1)	" "	" "	" "	" "	"*"	" "
7 (1)	"*"	"*"	"*"	" "	" "	" "
8 (1)	" "	" "	"*"	"*"	" "	"*"

	LeagueN	DivisionW	PutOuts	Assists	Errors
1 (1)	" "	" "	" "	" "	" "
2 (1)	" "	" "	" "	" "	" "
3 (1)	" "	" "	"*"	" "	" "
4 (1)	" "	"*"	"*"	" "	" "
5 (1)	" "	"*"	"*"	" "	" "
6 (1)	" "	"*"	"*"	" "	" "
7 (1)	" "	"*"	"*"	" "	" "
8 (1)	" "	"*"	"*"	" "	" "

	NewLeagueN
1 (1)	" "
2 (1)	" "

```
3 ( 1 ) " "  
4 ( 1 ) " "  
5 ( 1 ) " "  
6 ( 1 ) " "  
7 ( 1 ) " "  
8 ( 1 ) " "
```

Hide

```
model2<-regsubsets(Salary~.,data=hitters,nvmax=19)  
summary(model2)
```

Subset selection object

Call: regsubsets.formula(Salary ~ ., data = hitters, nvmax = 19)

19 Variables (and intercept)

	Forced in	Forced out
AtBat	FALSE	FALSE
Hits	FALSE	FALSE
HmRun	FALSE	FALSE
Runs	FALSE	FALSE
RBI	FALSE	FALSE
Walks	FALSE	FALSE
Years	FALSE	FALSE
CatBat	FALSE	FALSE
CHits	FALSE	FALSE
CHmRun	FALSE	FALSE
CRuns	FALSE	FALSE
CRBI	FALSE	FALSE
CWalks	FALSE	FALSE
LeagueN	FALSE	FALSE
DivisionW	FALSE	FALSE
PutOuts	FALSE	FALSE
Assists	FALSE	FALSE
Errors	FALSE	FALSE
NewLeagueN	FALSE	FALSE

1 subsets of each size up to 19

Selection Algorithm: exhaustive

		AtBat	Hits	HmRun	Runs	RBI	Walks	Years
1	(1)	" "	" "	" "	" "	" "	" "	" "
2	(1)	" "	"*"	" "	" "	" "	" "	" "
3	(1)	" "	"*"	" "	" "	" "	" "	" "
4	(1)	" "	"*"	" "	" "	" "	" "	" "
5	(1)	"*"	"*"	" "	" "	" "	" "	" "
6	(1)	"*"	"*"	" "	" "	" "	"*"	" "
7	(1)	" "	"*"	" "	" "	" "	"*"	" "
8	(1)	"*"	"*"	" "	" "	" "	"*"	" "
9	(1)	"*"	"*"	" "	" "	" "	"*"	" "
10	(1)	"*"	"*"	" "	" "	" "	"*"	" "
11	(1)	"*"	"*"	" "	" "	" "	"*"	" "
12	(1)	"*"	"*"	" "	"*"	" "	"*"	" "
13	(1)	"*"	"*"	" "	"*"	" "	"*"	" "
14	(1)	"*"	"*"	"*"	"*"	" "	"*"	" "
15	(1)	"*"	"*"	"*"	"*"	" "	"*"	" "
16	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "
17	(1)	"*"	"*"	"*"	"*"	"*"	"*"	" "
18	(1)	"*"	"*"	"*"	"*"	"*"	"*"	"*"
19	(1)	"*"	"*"	"*"	"*"	"*"	"*"	"*"

		CatBat	CHits	CHmRun	CRuns	CRBI	CWalks
1	(1)	" "	" "	" "	" "	"*"	" "
2	(1)	" "	" "	" "	" "	"*"	" "
3	(1)	" "	" "	" "	" "	"*"	" "
4	(1)	" "	" "	" "	" "	"*"	" "
5	(1)	" "	" "	" "	" "	"*"	" "
6	(1)	" "	" "	" "	" "	"*"	" "
7	(1)	"*"	"*"	"*"	" "	" "	" "
8	(1)	" "	" "	"*"	"*"	" "	"*"
9	(1)	"*"	" "	" "	"*"	"*"	"*"

```

10 ( 1 ) "*" " " " " "*" "*" "*"
11 ( 1 ) "*" " " " " "*" "*" "*"
12 ( 1 ) "*" " " " " "*" "*" "*"
13 ( 1 ) "*" " " " " "*" "*" "*"
14 ( 1 ) "*" " " " " "*" "*" "*"
15 ( 1 ) "*" "*" " " "*" "*" "*"
16 ( 1 ) "*" "*" " " "*" "*" "*"
17 ( 1 ) "*" "*" " " "*" "*" "*"
18 ( 1 ) "*" "*" " " "*" "*" "*"
19 ( 1 ) "*" "*" "*" "*" "*" "*"

```

LeagueN DivisionW PutOuts Assists Errors

```

1 ( 1 ) " " " " " " " "
2 ( 1 ) " " " " " " " "
3 ( 1 ) " " " " "*" " " "
4 ( 1 ) " " "*" "*" " " " "
5 ( 1 ) " " "*" "*" " " " "
6 ( 1 ) " " "*" "*" " " " "
7 ( 1 ) " " "*" "*" " " " "
8 ( 1 ) " " "*" "*" " " " "
9 ( 1 ) " " "*" "*" " " " "
10 ( 1 ) " " "*" "*" "*" " "
11 ( 1 ) "*" "*" "*" "*" " "
12 ( 1 ) "*" "*" "*" "*" " "
13 ( 1 ) "*" "*" "*" "*" "*"
14 ( 1 ) "*" "*" "*" "*" "*"
15 ( 1 ) "*" "*" "*" "*" "*"
16 ( 1 ) "*" "*" "*" "*" "*"
17 ( 1 ) "*" "*" "*" "*" "*"
18 ( 1 ) "*" "*" "*" "*" "*"
19 ( 1 ) "*" "*" "*" "*" "*"

```

NewLeagueN

```

1 ( 1 ) " "
2 ( 1 ) " "
3 ( 1 ) " "
4 ( 1 ) " "
5 ( 1 ) " "
6 ( 1 ) " "
7 ( 1 ) " "
8 ( 1 ) " "
9 ( 1 ) " "
10 ( 1 ) " "
11 ( 1 ) " "
12 ( 1 ) " "
13 ( 1 ) " "
14 ( 1 ) " "
15 ( 1 ) " "
16 ( 1 ) " "
17 ( 1 ) "*"
18 ( 1 ) "*"
19 ( 1 ) "*"

```

Hide

```
names(summary(model2))
```

```
[1] "which"  "rsq"    "rss"    "adjr2"  "cp"  
[6] "bic"    "outmat" "obj"
```

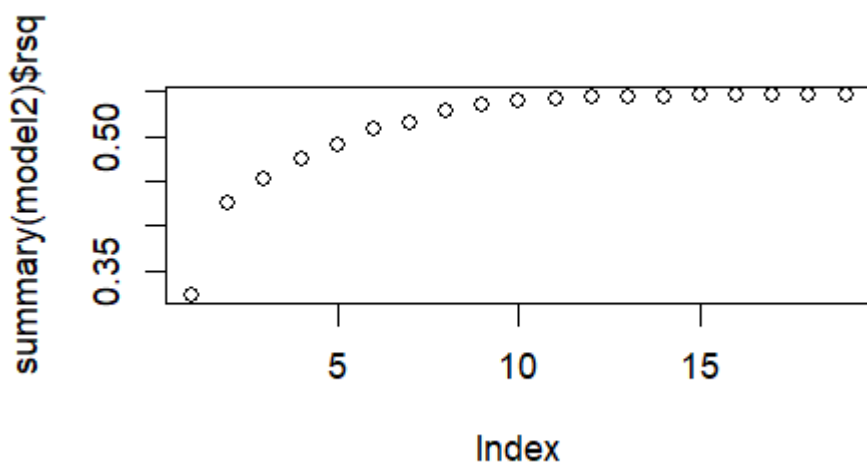
Hide

```
summary(model2)$rsq
```

```
[1] 0.3214501 0.4252237 0.4514294 0.4754067  
[5] 0.4908036 0.5087146 0.5141227 0.5285569  
[9] 0.5346124 0.5404950 0.5426153 0.5436302  
[13] 0.5444570 0.5452164 0.5454692 0.5457656  
[17] 0.5459518 0.5460945 0.5461159
```

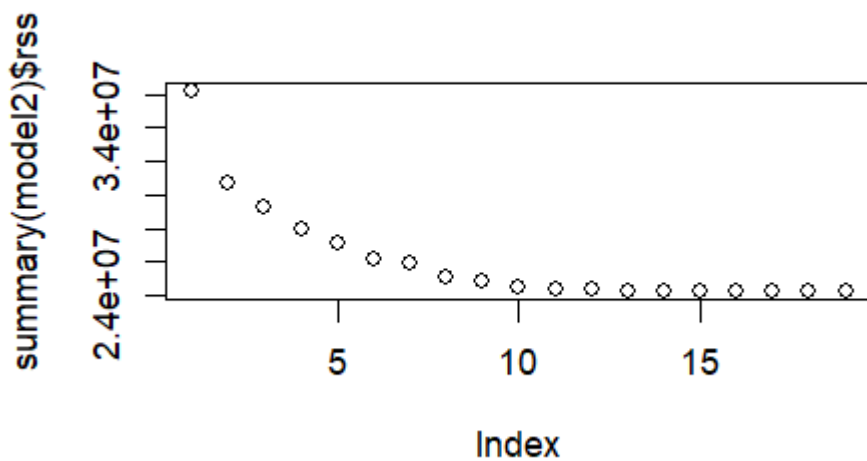
Hide

```
plot(summary(model2)$rsq)
```



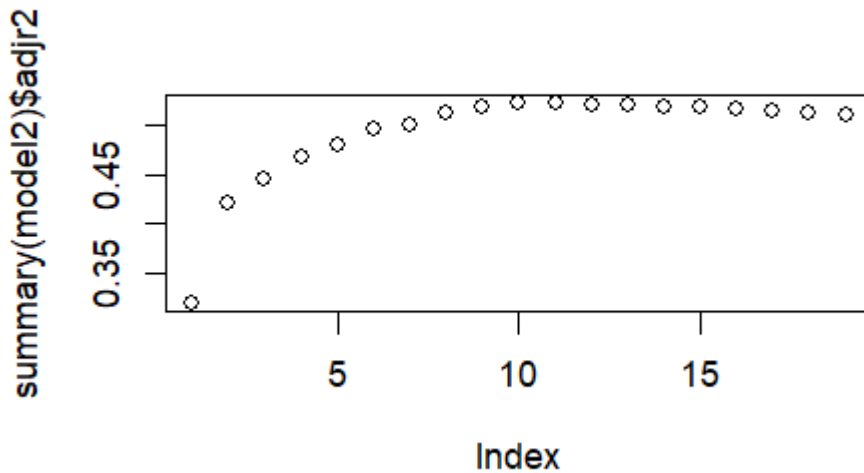
Hide

```
plot(summary(model2)$rss)
```



[Hide](#)

```
plot(summary(model2)$adjr2)
```

[Hide](#)

```
which.max(summary(model2)$adjr2)
```

```
[1] 11
```

[Hide](#)

```
coef(model2,11)
```

(Intercept)	AtBat	Hits
135.7512195	-2.1277482	6.9236994
Walks	CAtBat	CRuns
5.6202755	-0.1389914	1.4553310
CRBI	CWalks	LeagueN
0.7852528	-0.8228559	43.1116152
DivisionW	PutOuts	Assists
-111.1460252	0.2894087	0.2688277

The figures indicate that R-squared increase as the number of variables in the subset increases and likewise the residual sum of squared (sum of squared errors) decreases as the size of the subsets increases. On the other hand the adjusted R-squared increases first and then decreases.

Forward stepwise selection: In this example, the best model identified by the forward stepwise selection is the same as that obtained by the best subset selection. It is also possible to run this algorithm using a backward method where you drop variables one a time rather add. In general, the solutions from these two methods can be different.

[Hide](#)

```
model3<-regsubsets(Salary~.,data=hitters,nvmax=19,method="forward")
which.max(summary(model3)$adjr2)
```

```
[1] 11
```

[Hide](#)

```
coef(model3,11)
```

(Intercept)	AtBat	Hits
135.7512195	-2.1277482	6.9236994
Walks	CAtBat	CRuns
5.6202755	-0.1389914	1.4553310
CRBI	CWalks	LeagueN
0.7852528	-0.8228559	43.1116152
DivisionW	PutOuts	Assists
-111.1460252	0.2894087	0.2688277

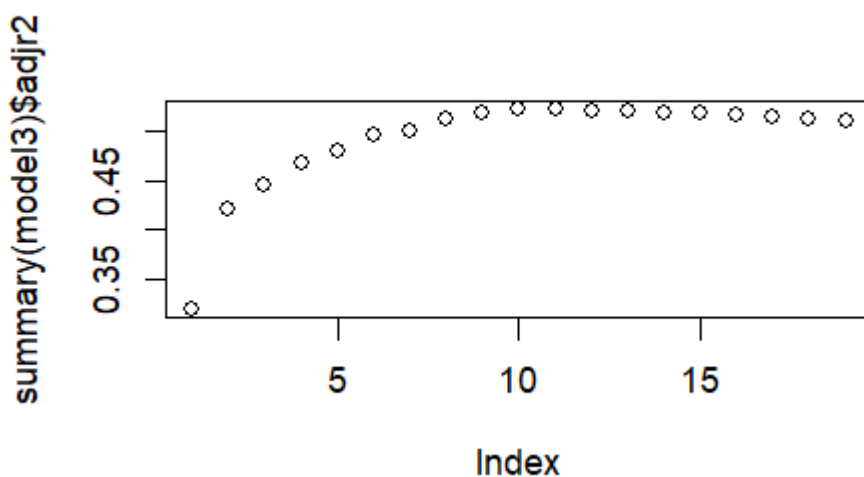
[Hide](#)

```
summary(model2)$adjr2-summary(model3)$adjr2
```

```
[1] 3.330669e-16 1.110223e-16 0.000000e+00  
[4] 0.000000e+00 1.110223e-16 0.000000e+00  
[7] 9.185854e-04 4.314850e-04 1.110223e-16  
[10] 1.110223e-16 1.110223e-16 0.000000e+00  
[13] 0.000000e+00 2.220446e-16 1.110223e-16  
[16] 0.000000e+00 0.000000e+00 0.000000e+00  
[19] 0.000000e+00
```

[Hide](#)

```
plot(summary(model3)$adjr2)
```

[Hide](#)

```
model4<-regsubsets(Salary~.,data=hitters,nvmax=19,method="backward")  
which.max(summary(model4)$adjr2)
```

```
[1] 11
```

[Hide](#)

```
coef(model3,11)
```

(Intercept)	AtBat	Hits
135.7512195	-2.1277482	6.9236994
Walks	CAtBat	CRuns
5.6202755	-0.1389914	1.4553310
CRBI	CWalks	LeagueN
0.7852528	-0.8228559	43.1116152
DivisionW	PutOuts	Assists
-111.1460252	0.2894087	0.2688277

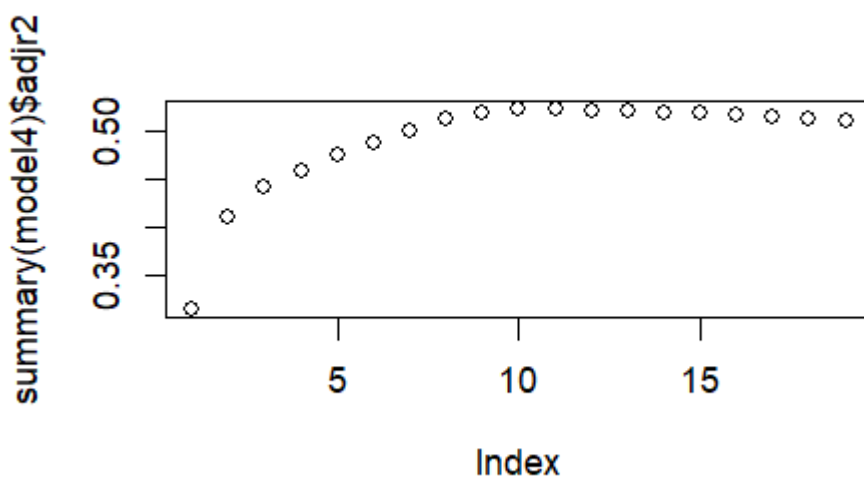
[Hide](#)

```
summary(model2)$adjr2-summary(model4)$adjr2
```

```
[1] 4.862441e-03 1.052500e-02 2.997619e-03
[4] 9.141119e-03 6.875981e-03 9.197808e-03
[7] 5.191286e-04 4.314850e-04 1.110223e-16
[10] 2.220446e-16 2.220446e-16 0.000000e+00
[13] -1.110223e-16 2.220446e-16 1.110223e-16
[16] 0.000000e+00 0.000000e+00 0.000000e+00
[19] 0.000000e+00
```

[Hide](#)

```
plot(summary(model4)$adjr2)
```



The following is AI generated code to help with making the plots fancier.

[Hide](#)

```
par(mfrow = c(1, 3))

plot(summary(model2)$adjr2, type = "b", xlab = "# Variables", ylab = "Adj. R²",
     main = "Exhaustive", pch = 20)
abline(v = which.max(summary(model2)$adjr2), col = "red", lty = 2)
```

Hide

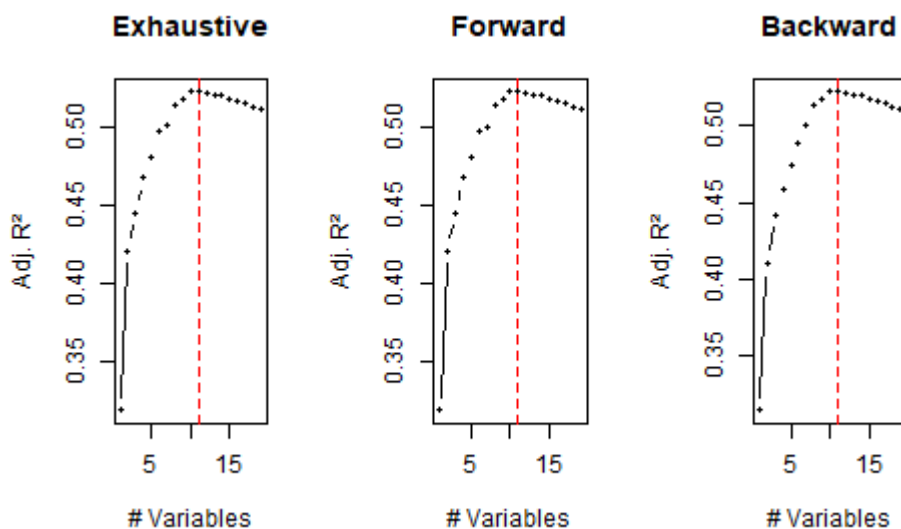
```
plot(summary(model3)$adjr2, type = "b", xlab = "# Variables", ylab = "Adj. R²",
     main = "Forward", pch = 20)
abline(v = which.max(summary(model3)$adjr2), col = "red", lty = 2)
```

Hide

```
plot(summary(model4)$adjr2, type = "b", xlab = "# Variables", ylab = "Adj. R²",
     main = "Backward", pch = 20)
abline(v = which.max(summary(model4)$adjr2), col = "red", lty = 2)
```

Hide

```
par(mfrow = c(1, 1))
```



LASSO

The generalized linear model with penalized maximum likelihood package `glmnet` in R implements the LASSO method. To run the `glmnet()` function, we need to pass in the arguments as `X` (input matrix), `y` (output vector), rather than the `y~X` format that we used thus far. The `model.matrix()` function produces a matrix corresponding to the 19 predictors and the intercept and helps transform qualitative variables into dummy quantitative variables. This is important since `glmnet()` works only with quantitative variables. We will work with the `X` matrix directly from this point onwards and get the solutions. We will also drop the first column corresponding to the intercept term that is introduced in `X` since `glmnet()` by default estimates the intercept and outputs it.

Hide

```
#install.packages("glmnet")
library(glmnet)
X <- model.matrix(Salary ~ ., data = hitters)[, -1]
# drop intercept column
str(X)
```

```
num [1:263, 1:19] 315 479 496 321 594 185 298 323 401 574 ...
- attr(*, "dimnames")=List of 2
..$ : chr [1:263] "2" "3" "4" "5" ...
..$ : chr [1:19] "AtBat" "Hits" "HmRun" "Runs" ...
```

Hide

```
colnames(X)
```

```
[1] "AtBat"      "Hits"       "HmRun"
[4] "Runs"       "RBI"        "Walks"
[7] "Years"      "CAtBat"     "CHits"
[10] "CHmRun"     "CRuns"      "CRBI"
[13] "CWalks"     "LeagueN"    "DivisionW"
[16] "PutOuts"    "Assists"    "Errors"
[19] "NewLeagueN"
```

Hide

```
y <- hitters$Salary
str(X)
```

```
num [1:263, 1:19] 315 479 496 321 594 185 298 323 401 574 ...
- attr(*, "dimnames")=List of 2
..$ : chr [1:263] "2" "3" "4" "5" ...
..$ : chr [1:19] "AtBat" "Hits" "HmRun" "Runs" ...
```

We now choose a range for the lambda parameters and create a training and test set. We then build the LASSO model on this data. The output from the model provides the Df (non of nonzeros), %Dev and Lambda values. The deviance measure is given as $2(\text{loglike_sat} - \text{loglike})$, where loglike_sat is the log-likelihood for the saturated model (a model with a free parameter per observation) and loglike is the log-likelihood of your model. Smaller the deviance, better the fit to the data. The deviance measure for linear regression is exactly the sum of squared errors. Null deviance is defined to be $2(\text{loglike_sat} - \text{loglike}(\text{NULL}))$ where the NULL model refers to the intercept model only. The null deviance for linear regression is exactly total sum of squares. The deviance ratio is $\text{dev.ratio} = 1 - \text{deviance} / \text{nulldev}$. You can interpret the dev.ratio simply as the Rsquared value for regression problems (this will be sufficient for our analysis). Higher the deviance ratio, better the fit to the data. As lambda decreases, the dev.ratio increases (more importance given to model fit than model complexity). The a0 output gives the intercept estimate and beta gives the beta estimates. We run glmnet here with the default lambda values. We can specify them too if we want.

Hide

```
set.seed(1)
train <- sample(1:nrow(X),nrow(X)/2)
test <- -train
modellasso <- glmnet(X[train,],y[train])
summary(modellasso)
```

	Length	Class	Mode
a0	100	-none-	numeric
beta	1900	dgCMatrix	S4
df	100	-none-	numeric
dim	2	-none-	numeric
lambda	100	-none-	numeric
dev.ratio	100	-none-	numeric
nulldev	1	-none-	numeric
npasses	1	-none-	numeric
jerr	1	-none-	numeric
offset	1	-none-	logical
call	3	-none-	call
nobs	1	-none-	numeric

[Hide](#)

```
modellasso
```

Call: glmnet(x = X[train,], y = y[train])

	Df	%Dev	Lambda
1	0	0.00	264.500
2	1	6.57	241.000
3	1	12.03	219.600
4	2	16.64	200.100
5	2	20.51	182.300
6	3	24.00	166.100
7	3	27.88	151.400
8	3	31.11	137.900
9	3	33.78	125.700
10	3	36.00	114.500
11	3	37.85	104.300
12	3	39.38	95.050
13	4	40.72	86.610
14	4	42.28	78.920
15	6	43.60	71.910
16	6	44.73	65.520
17	6	45.67	59.700
18	7	46.55	54.390
19	8	47.33	49.560
20	8	48.00	45.160
21	8	48.55	41.150
22	10	49.02	37.490
23	8	49.43	34.160
24	8	49.78	31.130
25	9	50.14	28.360
26	9	50.47	25.840
27	10	50.81	23.550
28	10	51.10	21.450
29	10	51.35	19.550
30	10	51.55	17.810
31	10	51.72	16.230
32	11	51.95	14.790
33	11	52.36	13.470
34	11	52.70	12.280
35	11	52.99	11.190
36	10	53.21	10.190
37	10	53.38	9.287
38	10	53.52	8.462
39	10	53.63	7.710
40	11	53.74	7.025
41	11	53.87	6.401
42	12	54.09	5.832
43	12	54.28	5.314
44	12	54.44	4.842
45	12	54.57	4.412
46	13	54.73	4.020
47	13	54.91	3.663
48	13	55.06	3.338
49	13	55.19	3.041
50	13	55.29	2.771
51	14	55.44	2.525

52	14	55.87	2.300
53	14	56.25	2.096
54	15	56.57	1.910
55	15	56.86	1.740
56	15	57.10	1.586
57	15	57.30	1.445
58	15	57.47	1.316
59	16	57.61	1.199
60	17	57.73	1.093
61	17	57.85	0.996
62	17	57.96	0.907
63	17	58.05	0.827
64	18	58.14	0.753
65	17	58.20	0.686
66	17	58.25	0.625
67	16	58.29	0.570
68	16	58.32	0.519
69	16	58.34	0.473
70	16	58.36	0.431
71	16	58.38	0.393
72	17	58.40	0.358
73	17	58.41	0.326
74	17	58.42	0.297
75	19	58.44	0.271
76	19	58.45	0.247
77	19	58.49	0.225
78	19	58.49	0.205
79	19	58.52	0.187
80	19	58.53	0.170
81	19	58.54	0.155
82	19	58.55	0.141
83	19	58.56	0.129
84	19	58.57	0.117
85	19	58.58	0.107
86	19	58.58	0.097
87	19	58.59	0.089
88	19	58.59	0.081
89	19	58.59	0.074
90	19	58.60	0.067
91	19	58.60	0.061
92	19	58.60	0.056
93	19	58.60	0.051
94	19	58.60	0.046
95	19	58.61	0.042
96	19	58.61	0.038
97	19	58.61	0.035
98	19	58.61	0.032
99	19	58.61	0.029
100	19	58.61	0.026

[Hide](#)

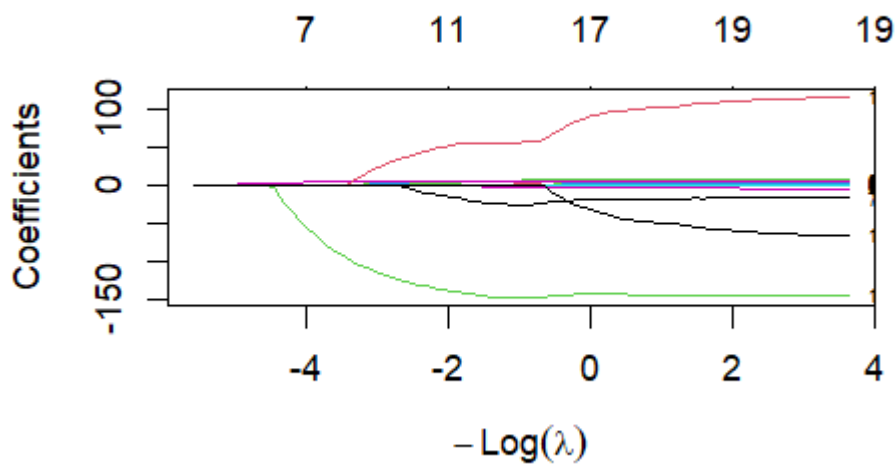
```
deviance(modellasso)
```



```
[1] 23667021 22111050 20819255 19729261 18812379
[6] 17987594 17067936 16305047 15671748 15145887
[11] 14709390 14346937 14030529 13660055 13349004
[16] 13080381 12857190 12648873 12465519 12307920
[21] 12176866 12065305 11967669 11886379 11799332
[26] 11721223 11642503 11573553 11514204 11466000
[31] 11425790 11371598 11275818 11193426 11124830
[36] 11074639 11034246 11000792 10973373 10948995
[41] 10918261 10865633 10820567 10783366 10752548
[46] 10714120 10671010 10635053 10605317 10580452
[51] 10546091 10443113 10353636 10277597 10209628
[56] 10153039 10105839 10066378 10032826 10004488
[61] 9976542 9949474 9927139 9907822 9891754
[66] 9881002 9872464 9865121 9859088 9853913
[71] 9849692 9846101 9842712 9840754 9836985
[76] 9833276 9824988 9823291 9816430 9815432
[81] 9812125 9809295 9807128 9805331 9803845
[86] 9802571 9801484 9800562 9799763 9799074
[91] 9798466 9797966 9797527 9797128 9796796
[96] 9796509 9796249 9796028 9795842 9795662
```

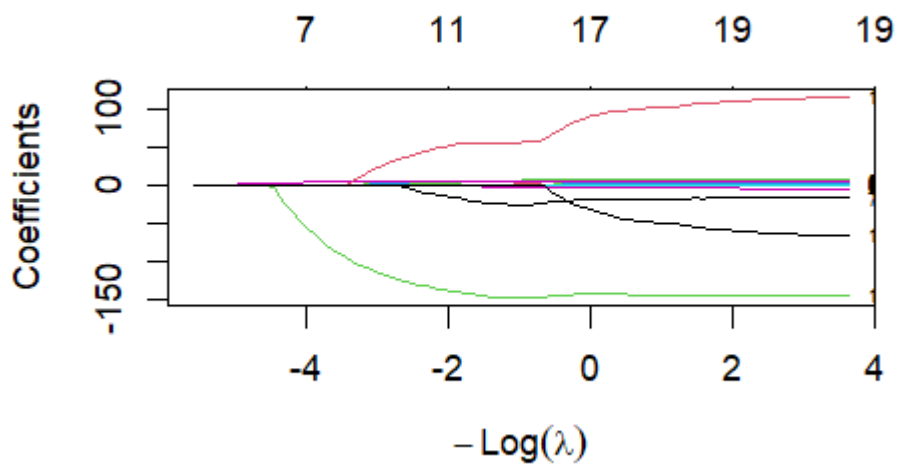
Hide

```
plot(modellasso,label=TRUE)
```



Hide

```
plot(modellasso,xvar="lambda",label=TRUE)
```



We see from the plot that as λ increases, many of the coefficients get close to zero. We can retrieve these coefficients at specific values of λ . The a_0 term keeps track of the intercept in glmnet. Note that the number of non-zero coefficients does not change in a fully monotonic way, as λ increases or decreases but the general trend is that as λ increases, the number of zero coefficients increases (or number of non-zero coefficients decrease).

Hide

```
modellasso$df
```

```
[1]  0  1  1  2  2  3  3  3  3  3  3  3  4  4  6
[16]  6  6  7  8  8  8 10  8  8  9  9 10 10 10 10
[31] 10 11 11 11 11 10 10 10 10 11 11 12 12 12 12
[46] 13 13 13 13 13 14 14 14 15 15 15 15 15 16 17
[61] 17 17 17 18 17 17 16 16 16 16 16 17 17 17 19
[76] 19 19 19 19 19 19 19 19 19 19 19 19 19 19 19
[91] 19 19 19 19 19 19 19 19 19 19 19
```

Hide

```
modellasso$a0
```

s0	s1	s2	s3	s4
531.86537	505.49838	481.47375	460.51767	442.09143
s5	s6	s7	s8	s9
421.58253	389.38027	359.92818	333.07966	308.63793
s10	s11	s12	s13	s14
286.34755	266.05749	248.41927	239.17924	230.00988
s15	s16	s17	s18	s19
219.78200	210.38800	200.74720	190.94915	181.34000
s20	s21	s22	s23	s24
172.72411	164.64615	157.17910	150.17758	140.73484
s25	s26	s27	s28	s29
131.16210	122.46907	114.90372	107.80904	101.40317
s30	s31	s32	s33	s34
95.54681	93.53147	100.64108	107.72647	114.28785
s35	s36	s37	s38	s39
117.62780	119.84177	121.90781	123.69211	125.72654
s40	s41	s42	s43	s44
129.28822	140.89645	151.71947	161.52186	170.42527
s45	s46	s47	s48	s49
179.89838	189.90348	198.97075	207.19205	214.71282
s50	s51	s52	s53	s54
222.11509	225.18344	227.97429	230.39376	232.80785
s55	s56	s57	s58	s59
234.98599	236.97476	238.72409	240.73568	242.04725
s60	s61	s62	s63	s64
243.48182	246.08704	248.51711	250.79599	253.43746
s65	s66	s67	s68	s69
256.16948	258.59577	260.59389	262.58883	264.23261
s70	s71	s72	s73	s74
265.81974	267.26457	268.60895	269.52522	270.63066
s75	s76	s77	s78	s79
271.37281	270.94552	271.55508	271.42364	271.85351
s80	s81	s82	s83	s84
272.69812	273.02675	273.21919	273.37165	273.50387
s85	s86	s87	s88	s89
273.62310	273.73075	273.82844	273.91767	273.99861
s90	s91	s92	s93	s94
274.07226	274.13785	274.19868	274.25473	274.30409
s95	s96	s97	s98	s99
274.34926	274.39121	274.42860	274.46225	274.49476

[Hide](#)

```
as.matrix(modellasso$beta)
```

	s0	s1	s2	s3
AtBat	0 0.00000000	0.00000000	0.00000000	
Hits	0 0.00000000	0.00000000	0.00000000	
HmRun	0 0.00000000	0.00000000	0.00000000	
Runs	0 0.00000000	0.00000000	0.00000000	
RBI	0 0.00000000	0.00000000	0.00000000	
Walks	0 0.00000000	0.00000000	0.00000000	
Years	0 0.00000000	0.00000000	0.00000000	
CAtBat	0 0.00000000	0.00000000	0.00000000	
CHits	0 0.00000000	0.00000000	0.00000000	
CHmRun	0 0.00000000	0.00000000	0.0877153	
	s4	s5	s6	s7
AtBat	0.00000000	0.00000000	0.00000000	0.00000000
Hits	0.00000000	0.00000000	0.00000000	0.00000000
HmRun	0.00000000	0.00000000	0.00000000	0.00000000
Runs	0.00000000	0.00000000	0.00000000	0.00000000
RBI	0.00000000	0.00000000	0.00000000	0.00000000
Walks	0.00000000	0.1095612	0.6210547	1.0880248
Years	0.00000000	0.00000000	0.00000000	0.00000000
CAtBat	0.00000000	0.00000000	0.00000000	0.00000000
CHits	0.00000000	0.00000000	0.00000000	0.00000000
CHmRun	0.2303683	0.3429256	0.4232717	0.4888717
	s8	s9	s10	s11
AtBat	0.00000000	0.00000000	0.00000000	0.00000000
Hits	0.00000000	0.00000000	0.00000000	0.00000000
HmRun	0.00000000	0.00000000	0.00000000	0.00000000
Runs	0.00000000	0.00000000	0.00000000	0.00000000
RBI	0.00000000	0.00000000	0.00000000	0.00000000
Walks	1.5136197	1.9012226	2.2545612	2.5763397
Years	0.00000000	0.00000000	0.00000000	0.00000000
CAtBat	0.00000000	0.00000000	0.00000000	0.00000000
CHits	0.00000000	0.00000000	0.00000000	0.00000000
CHmRun	0.5477657	0.6029035	0.6517801	0.6976866
	s12	s13	s14	
AtBat	0.00000000	0.00000000	0.0000000000	
Hits	0.00000000	0.00000000	0.0000000000	
HmRun	0.00000000	0.00000000	0.0000000000	
Runs	0.00000000	0.00000000	0.007804832	
RBI	0.00000000	0.00000000	0.023898403	
Walks	2.8723279	3.1556696	3.393837658	
Years	0.00000000	0.00000000	0.0000000000	
CAtBat	0.00000000	0.00000000	0.0000000000	
CHits	0.00000000	0.00000000	0.0000000000	
CHmRun	0.7317752	0.7632056	0.778216900	
	s15	s16	s17	
AtBat	0.00000000	0.00000000	0.0000000000	
Hits	0.00000000	0.00000000	0.0000000000	
HmRun	0.00000000	0.00000000	0.0000000000	
Runs	0.08093599	0.1516809	0.214117019	
RBI	0.07270051	0.1151076	0.141388496	
Walks	3.54052128	3.6724799	3.758336099	
Years	0.00000000	0.00000000	0.0000000000	
CAtBat	0.00000000	0.00000000	0.0000000000	
CHits	0.00000000	0.00000000	0.0000000000	
CHmRun	0.78330289	0.7858554	0.780583983	

	s18	s19	s20
AtBat	0.00000000	0.00000000	0.00000000
Hits	0.01498891	0.1186950	0.21745394
HmRun	0.00000000	0.00000000	0.00000000
Runs	0.26661387	0.1786221	0.09732585
RBI	0.15083828	0.1236041	0.09019558
Walks	3.81326488	3.8775423	3.93628016
Years	0.00000000	0.00000000	0.00000000
CAtBat	0.00000000	0.00000000	0.00000000
CHits	0.00000000	0.00000000	0.00000000
CHmRun	0.76592306	0.7944974	0.83427366
	s21	s22	s23
AtBat	0.000000000	0.000000000	0.000000000
Hits	0.295875262	0.33910800	0.34967540
HmRun	0.064053026	0.39035097	0.56712264
Runs	0.012301833	0.000000000	0.000000000
RBI	0.072512331	0.000000000	0.000000000
Walks	3.998237124	4.04263883	4.08292259
Years	0.000000000	0.000000000	0.000000000
CAtBat	0.000000000	0.000000000	0.000000000
CHits	0.005243442	0.01252331	0.02693188
CHmRun	0.871725632	0.87222074	0.91414101
	s24	s25	
AtBat	0.000000000	0.000000000	
Hits	0.37995494	0.41376625	
HmRun	0.72709544	0.87818639	
Runs	0.000000000	0.000000000	
RBI	0.000000000	0.000000000	
Walks	4.08592669	4.07837663	
Years	0.000000000	0.000000000	
CAtBat	0.000000000	0.000000000	
CHits	0.03973360	0.04971208	
CHmRun	0.96019641	0.99290548	
	s26	s27	
AtBat	0.000000000	0.000000000	
Hits	0.41171670	0.39018482	
HmRun	1.11310560	1.39778961	
Runs	0.000000000	0.000000000	
RBI	0.000000000	0.000000000	
Walks	4.07631819	4.08465641	
Years	0.000000000	0.000000000	
CAtBat	0.000000000	0.000000000	
CHits	0.06032560	0.06582110	
CHmRun	1.04081759	1.04924490	
	s28	s29	
AtBat	0.000000000	0.000000000	
Hits	0.36119995	0.34303485	
HmRun	1.70277323	1.94107401	
Runs	0.000000000	0.000000000	
RBI	0.000000000	0.000000000	
Walks	4.09744804	4.10348570	
Years	0.000000000	0.000000000	
CAtBat	0.000000000	0.000000000	
CHits	0.07683496	0.08378951	
CHmRun	1.07732680	1.09844269	
	s30	s31	

AtBat	0.00000000	0.00000000	
Hits	0.32556062	0.29143832	
HmRun	2.16254406	2.37477111	
Runs	0.00000000	0.00000000	
RBI	0.00000000	0.00000000	
Walks	4.10949673	4.11106813	
Years	0.00000000	-0.76442030	
CAtBat	0.00000000	0.00000000	
CHits	0.09073734	0.10450327	
CHmRun	1.11988415	1.14006395	
	s32	s33	s34
AtBat	0.00000000	0.00000000	0.00000000
Hits	0.2029085	0.1119611	0.02659006
HmRun	2.6600735	2.9678217	3.26481054
Runs	0.00000000	0.00000000	0.00000000
RBI	0.00000000	0.00000000	0.00000000
Walks	4.1099380	4.1136122	4.11875476
Years	-3.4505573	-6.0200626	-8.37700568
CAtBat	0.00000000	0.00000000	0.00000000
CHits	0.1309559	0.1550448	0.17600665
CHmRun	1.0987462	1.0351605	0.96360902
	s35	s36	s37
AtBat	0.00000000	0.00000000	0.00000000
Hits	0.00000000	0.00000000	0.00000000
HmRun	3.3875312	3.4239422	3.4527540
Runs	0.00000000	0.00000000	0.00000000
RBI	0.00000000	0.00000000	0.00000000
Walks	4.0919122	4.0487555	4.0095547
Years	-10.2055368	-11.8252122	-13.2996313
CAtBat	0.00000000	0.00000000	0.00000000
CHits	0.1918772	0.2052751	0.2172280
CHmRun	0.9298302	0.9106627	0.8938979
	s38	s39	s40
AtBat	0.00000000	0.00000000	0.00000000
Hits	0.00000000	0.00000000	0.00000000
HmRun	3.4805209	3.50583637	3.5819252
Runs	0.00000000	0.00000000	0.00000000
RBI	0.00000000	0.00000000	0.00000000
Walks	3.9747491	3.94331284	3.9172989
Years	-14.6169107	-15.86828408	-17.0263001
CAtBat	0.00000000	0.00000000	0.00000000
CHits	0.2278926	0.23764801	0.2460677
CHmRun	0.8789078	0.86167343	0.8429630
	s41	s42	
AtBat	-0.04389081	-0.08706944	
Hits	0.00000000	0.00000000	
HmRun	4.02705026	4.47870279	
Runs	0.00000000	0.00000000	
RBI	0.00000000	0.00000000	
Walks	3.98899742	4.06333631	
Years	-18.72567091	-20.25674145	
CAtBat	0.00000000	0.00000000	
CHits	0.25858285	0.26968276	
CHmRun	0.75620052	0.66500649	
	s43	s44	s45
AtBat	-0.1261800	-0.1616992	-0.2357964

Hits	0.0000000	0.0000000	0.1503145
HmRun	4.8880704	5.2595941	5.5215808
Runs	0.0000000	0.0000000	0.0000000
RBI	0.0000000	0.0000000	0.0000000
Walks	4.1309976	4.1925304	4.2394764
Years	-21.6387651	-22.8928771	-23.9331886
CAtBat	0.0000000	0.0000000	0.0000000
CHits	0.2794007	0.2881900	0.2931407
CHmRun	0.5809432	0.5047707	0.4581748
	s46	s47	s48
AtBat	-0.3430320	-0.4417090	-0.5310141
Hits	0.4207553	0.6717305	0.8988989
HmRun	5.7201031	5.8932897	6.0488698
Runs	0.0000000	0.0000000	0.0000000
RBI	0.0000000	0.0000000	0.0000000
Walks	4.2824817	4.3211871	4.3560846
Years	-24.7157142	-25.4057224	-26.0347130
CAtBat	0.0000000	0.0000000	0.0000000
CHits	0.2937731	0.2942690	0.2949078
CHmRun	0.4350495	0.4185656	0.4051273
	s49	s50	
AtBat	-0.6130936	-6.974205e-01	
Hits	1.1080601	1.316676e+00	
HmRun	6.1905047	6.365296e+00	
Runs	0.0000000	0.000000e+00	
RBI	0.0000000	0.000000e+00	
Walks	4.3880229	4.422774e+00	
Years	-26.6049110	-2.688074e+01	
CAtBat	0.0000000	-5.529584e-03	
CHits	0.2954473	3.117413e-01	
CHmRun	0.3933633	3.757822e-01	
	s51	s52	
AtBat	-0.68676536	-0.67328543	
Hits	1.18967443	1.06104880	
HmRun	6.57606149	6.75337728	
Runs	0.00000000	0.00000000	
RBI	0.00000000	0.00000000	
Walks	4.42752733	4.42820350	
Years	-25.75316318	-24.69553852	
CAtBat	-0.05384494	-0.09998821	
CHits	0.48643509	0.65507480	
CHmRun	0.50133718	0.63765618	
	s53	s54	s55
AtBat	-0.6586566	-0.6419337	-0.6265019
Hits	0.9377095	0.8158301	0.7043822
HmRun	6.9073613	7.0290653	7.1393125
Runs	0.0000000	0.0000000	0.0000000
RBI	0.0000000	0.0000000	0.0000000
Walks	4.4297979	4.4371295	4.4436889
Years	-23.6881389	-22.7103271	-21.8180747
CAtBat	-0.1428055	-0.1836075	-0.2207736
CHits	0.8116446	0.9610909	1.0972308
CHmRun	0.7672472	0.8951414	1.0119011
	s56	s57	s58
AtBat	-0.6124100	-0.5990622	-5.870480e-01
Hits	0.6026829	0.5087318	4.269444e-01

HmRun	7.2397771	7.3302901	7.431346e+00
Runs	0.0000000	0.0000000	0.000000e+00
RBI	0.0000000	0.0000000	0.000000e+00
Walks	4.4496497	4.4548539	4.435141e+00
Years	-21.0053472	-20.2529359	-1.962572e+01
CAtBat	-0.2546619	-0.2856818	-3.144890e-01
CHits	1.2213718	1.3350223	1.438727e+00
CHmRun	1.1183617	1.2166799	1.298378e+00

s59	s60
-----	-----

AtBat	-5.673185e-01	-0.54759522
Hits	3.289713e-01	0.22964936
HmRun	7.587644e+00	7.69580062
Runs	0.000000e+00	0.00000000
RBI	0.000000e+00	0.00000000
Walks	4.389496e+00	4.36436918
Years	-1.903680e+01	-18.46130111
CAtBat	-3.419655e-01	-0.36929810
CHits	1.534972e+00	1.63976579
CHmRun	1.345523e+00	1.42645912

s61	s62
-----	-----

AtBat	-0.53281982	-0.52039249
Hits	0.15194968	0.08504343
HmRun	7.78795214	7.86907401
Runs	0.00000000	0.00000000
RBI	0.00000000	0.00000000
Walks	4.34571978	4.32937912
Years	-18.25488089	-18.08592383
CAtBat	-0.39142736	-0.41106188
CHits	1.74866333	1.84654525
CHmRun	1.54409570	1.65236207

s63	s64
-----	-----

AtBat	-0.50955982	-0.51708614
Hits	0.01009352	0.00000000
HmRun	7.84880085	7.77757656
Runs	0.00000000	0.00000000
RBI	0.05585887	0.11674142
Walks	4.31015356	4.29501834
Years	-17.93596458	-17.79235548
CAtBat	-0.42936081	-0.44454064
CHits	1.94024494	2.02028507
CHmRun	1.76392428	1.87967472

s65	s66	s67
-----	-----	-----

AtBat	-5.245988e-01	-0.5301626	-0.5348956
Hits	0.000000e+00	0.0000000	0.0000000
HmRun	7.814991e+00	7.9021838	7.9862144
Runs	0.000000e+00	0.0000000	0.0000000
RBI	1.324328e-01	0.1258950	0.1182084
Walks	4.283094e+00	4.2722950	4.2636457
Years	-1.780288e+01	-17.8161205	-17.7739683
CAtBat	-4.556036e-01	-0.4656498	-0.4751680
CHits	2.073094e+00	2.1168830	2.1579801
CHmRun	1.917379e+00	1.9267525	1.9359981

s68	s69	s70
-----	-----	-----

AtBat	-0.5395601	-0.5434582	-0.5471831
Hits	0.0000000	0.0000000	0.0000000
HmRun	8.0589775	8.1292599	8.1907441

Runs	0.0000000	0.0000000	0.0000000
RBI	0.1124021	0.1057942	0.1008876
Walks	4.2551901	4.2480742	4.2412977
Years	-17.7759689	-17.7360789	-17.7271498
CAtBat	-0.4836757	-0.4916314	-0.4986073
CHits	2.1947560	2.2290690	2.2594441
CHmRun	1.9440216	1.9518105	1.9583886

s71	s72
-----	-----

AtBat	-5.506137e-01	-0.55680609
Hits	0.000000e+00	0.00000000
HmRun	8.247522e+00	8.28176237
Runs	9.917932e-04	0.03393223
RBI	9.549709e-02	0.08546903
Walks	4.234870e+00	4.22289714
Years	-1.771514e+01	-17.67612673
CAtBat	-5.050723e-01	-0.51126343
CHits	2.287517e+00	2.31540914
CHmRun	1.964632e+00	1.97485774

s73	s74
-----	-----

AtBat	-0.56070749	-5.583130e-01
Hits	0.00000000	-4.901684e-02
HmRun	8.31803987	8.328538e+00
Runs	0.07350006	1.300860e-01
RBI	0.06876119	7.488906e-02
Walks	4.20802316	4.192923e+00
Years	-17.73754880	-1.765420e+01
CAtBat	-0.51345109	-5.219481e-01
CHits	2.33127369	2.368084e+00
CHmRun	1.98069080	1.992236e+00

s75	s76
-----	-----

AtBat	-0.55057822	-0.52492002
Hits	-0.13611806	-0.29358911
HmRun	8.26403646	7.98605756
Runs	0.25752804	0.34054415
RBI	0.09675888	0.22265491
Walks	4.16173746	4.12308487
Years	-17.77094524	-17.42023461
CAtBat	-0.52516454	-0.54055157
CHits	2.40161863	2.48664582
CHmRun	2.02478360	2.18820733

s77	s78	s79
-----	-----	-----

AtBat	-0.52229530	-0.5001954	-0.4984410
Hits	-0.33818021	-0.4880850	-0.5190280
HmRun	7.95933849	7.6705790	7.6607036
Runs	0.41783578	0.5123787	0.5698516
RBI	0.23130401	0.3574294	0.3586530
Walks	4.10440608	4.0660645	4.0529725
Years	-17.53179529	-17.2711701	-17.3510711
CAtBat	-0.54155047	-0.5550739	-0.5556771
CHits	2.50370059	2.5844711	2.5960810
CHmRun	2.20448004	2.3649912	2.3740926

s80	s81	s82
-----	-----	-----

AtBat	-0.4916241	-0.4832827	-0.4753997
Hits	-0.6082420	-0.6855206	-0.7507864
HmRun	7.4946583	7.3442602	7.2295659
Runs	0.6836980	0.7602721	0.8198393

RBI	0.4241907	0.4849315	0.5328688
Walks	4.0214489	3.9986218	3.9795885
Years	-17.4575999	-17.4306900	-17.3867222
CAtBat	-0.5597045	-0.5649536	-0.5697249
CHits	2.6382255	2.6781677	2.7115317
CHmRun	2.4523323	2.5315217	2.5944710
	s83	s84	s85
AtBat	-0.4680592	-0.4614609	-0.4553907
Hits	-0.8095269	-0.8618238	-0.9097431
HmRun	7.1308939	7.0448661	6.9664515
Runs	0.8722186	0.9185587	0.9609293
RBI	0.5746379	0.6113412	0.6448167
Walks	3.9624250	3.9470957	3.9330420
Years	-17.3428070	-17.3030856	-17.2663992
CAtBat	-0.5740918	-0.5779910	-0.5815691
CHits	2.7413250	2.7677329	2.7919041
CHmRun	2.6493736	2.6975238	2.7414602
	s86	s87	s88
AtBat	-0.4498613	-0.4448708	-0.4402837
Hits	-0.9533190	-0.9926569	-1.0287913
HmRun	6.8953503	6.8313210	6.7724440
Runs	0.9994110	1.0341538	1.0660612
RBI	0.6751997	0.7026057	0.7277745
Walks	3.9202602	3.9087133	3.8981111
Years	-17.2328690	-17.2026976	-17.1749055
CAtBat	-0.5848256	-0.5877638	-0.5904641
CHits	2.8138731	2.8336927	2.8519034
CHmRun	2.7813344	2.8172701	2.8502985
	s89	s90	s91
AtBat	-0.4360972	-0.4321793	-0.4287865
Hits	-1.0617493	-1.0924995	-1.1192052
HmRun	6.7187427	6.6684885	6.6251740
Runs	1.0951459	1.1222285	1.1457657
RBI	0.7507277	0.7721351	0.7907181
Walks	3.8884455	3.8794443	3.8716135
Years	-17.1494366	-17.1252853	-17.1045126
CAtBat	-0.5929287	-0.5952344	-0.5972330
CHits	2.8685159	2.8840331	2.8974856
CHmRun	2.8804291	2.9086002	2.9329645
	s92	s93	s94
AtBat	-0.4256533	-0.4226520	-0.4200395
Hits	-1.1438749	-1.1674123	-1.1879332
HmRun	6.5850806	6.5465658	6.5131965
Runs	1.1675369	1.1882677	1.2063333
RBI	0.8078826	0.8242609	0.8385327
Walks	3.8643751	3.8574869	3.8514828
Years	-17.0854824	-17.0669776	-17.0507848
CAtBat	-0.5990772	-0.6008425	-0.6023816
CHits	2.9099134	2.9217939	2.9321419
CHmRun	2.9554836	2.9770585	2.9958214
	s95	s96	s97
AtBat	-0.4176825	-0.4154527	-0.4134777
Hits	-1.2064699	-1.2239677	-1.2394826
HmRun	6.4831026	6.4545554	6.4293285
Runs	1.2226686	1.2380793	1.2517457
RBI	0.8514226	0.8635921	0.8743754

```
Walks      3.8460524    3.8409297    3.8363917
Years     -17.0363031   -17.0225701   -17.0103557
CAtBat    -0.6037697    -0.6050816    -0.6062447
CHits      2.9414831    2.9503098    2.9581331
CHmRun     3.0127471    3.0287610    3.0429471
```

```
      s98      s99
```

```
AtBat     -0.4117550   -0.4100302
Hits      -1.2530631   -1.2665960
HmRun      6.4073664    6.3852223
Runs       1.2637304    1.2756560
RBI        0.8838123    0.8932321
Walks      3.8324146    3.8284479
Years     -16.9998147   -16.9892716
CAtBat    -0.6072602   -0.6082740
CHits      2.9649712    2.9717990
CHmRun     3.0553285    3.0677219
```

```
[ reached 'max' / getOption("max.print") -- omitted 9 rows ]
```

Hide

```
modellasso$lambda[1]
```

```
[1] 264.4958
```

Hide

```
modellasso$a0[1]
```

```
s0
```

```
531.8654
```

Hide

```
modellasso$beta[,1]
```

```
AtBat    Hits    HmRun    Runs
      0      0      0      0
RBI      Walks   Years    CAtBat
      0      0      0      0
CHits    CHmRun   CRuns    CRBI
      0      0      0      0
CWalks   LeagueN DivisionW PutOuts
      0      0      0      0
Assists  Errors  NewLeagueN
      0      0      0
```

Hide

```
modellasso$lambda[70]
```

```
[1] 0.4310623
```

[Hide](#)

```
modellasso$a0[70]
```

```
s69  
264.2326
```

[Hide](#)

```
modellasso$beta[,70]
```

AtBat	Hits	HmRun
-0.5434582	0.0000000	8.1292599
Runs	RBI	Walks
0.0000000	0.1057942	4.2480742
Years	CAtBat	CHits
-17.7360789	-0.4916314	2.2290690
CHmRun	CRuns	CRBI
1.9518105	-0.3334579	0.0000000
CWalks	LeagueN	DivisionW
0.1652172	102.0714817	-145.2585272
PutOuts	Assists	Errors
0.1885423	0.6043276	-3.6597910
NewLeagueN		
-48.8394407		

[Hide](#)

```
modellasso$lambda[100]
```

```
[1] 0.02644958
```

[Hide](#)

```
modellasso$a0[100]
```

```
s99  
274.4948
```

[Hide](#)

```
modellasso$beta[,100]
```

AtBat	Hits	HmRun
-0.4100302	-1.2665960	6.3852223
Runs	RBI	Walks
1.2756560	0.8932321	3.8284479
Years	CAtBat	CHits
-16.9892716	-0.6082740	2.9717990
CHmRun	CRuns	CRBI
3.0677219	-0.8696309	-0.4612084
CWalks	LeagueN	DivisionW
0.3122121	115.8333379	-144.8891401
PutOuts	Assists	Errors
0.1957103	0.6683825	-4.5586077
NewLeagueN		
-66.8223277		

We can also return the coefficients using the `coef` command. It returns it for specific values of `lambda`. For values in the grid it returns the exact beta parameters while for values not in the grid, it returns values using interpolation between closest `lambda` values in the grid. You can use the `exact=T` with the training data to get the exact beta values.

Hide

```
coef(modellasso,s=15)
```

```
20 x 1 sparse Matrix of class "dgCMatrix"
               s=15
(Intercept)  93.82856496
AtBat        .
Hits         0.29646857
HmRun        2.34348494
Runs         .
RBI          .
Walks        4.11083648
Years       -0.65173068
CAtBat       .
CHits        0.10247392
CHmRun       1.13708907
CRuns        .
CRBI         0.34482924
CWalks       .
LeagueN     34.57411925
DivisionW   -123.77714349
PutOuts      0.10005119
Assists      0.09653002
Errors       .
NewLeagueN  .
```

Hide

```
coef(modellasso,s=15,exact=T,x=X[train,],y=y[train])
```

```
20 x 1 sparse Matrix of class "dgCMatrix"
```

```
s=15
```

```
(Intercept)  92.41848657
AtBat        .
Hits         0.30496059
HmRun        2.33686628
Runs         .
RBI          .
Walks        4.11219805
Years        -0.32925757
CAtBat       .
CHits        0.09898385
CHmRun       1.13566970
CRuns        .
CRBI         0.34694553
CWalks       .
LeagueN     34.60588585
DivisionW    -123.75235053
PutOuts      0.09986349
Assists      0.09655562
Errors       .
NewLeagueN   .
```

Predictions: We start with a prediction for the model fitted with $\lambda = 100$. The test mean squared error for this model is 151898.1. Suppose, we change λ to 50, we get 144989 and if we change λ to 200, we get 183137.7. Note that by default if prediction is done at λ values that are not tried in the fitting algorithm, it uses linear interpolation to make predictions. We can use `exact=T` in the argument to get the exact value by refitting. In addition, you need to then pass also the original training set data to the function. You get a test error of 167213.2 with the full model (all variables) while 224669.9 with a very large value of λ . Thus choosing λ appropriately will be important in the quality of the fit. This can be done with cross-validation.

[Hide](#)

```
modellasso$lambda
```

```
[1] 264.49580153 240.99872350 219.58906113
[4] 200.08137415 182.30669632 166.11107187
[7] 151.35422207 137.90833013 125.65693417
[10] 114.49391847 104.32259431 95.05486256
[13] 86.61045055 78.91621683 71.90551763
[16] 65.51762962 59.69722398 54.39388714
[19] 49.56168412 45.15876070 41.14698086
[22] 37.49159649 34.16094639 31.12618206
[25] 28.36101783 25.84150318 23.54581526
[28] 21.45406992 19.54814948 17.81154576
[31] 16.22921712 14.78745820 13.47378117
[34] 12.27680759 11.18616984 10.19242134
[37] 9.28695472 8.46192726 7.71019296
[40] 7.02524067 6.40113765 5.83247823
[43] 5.31433693 4.84222589 4.41205589
[46] 4.02010100 3.66296631 3.33755848
[49] 3.04105898 2.77089968 2.52474059
[52] 2.30044960 2.09608399 1.90987367
[55] 1.74020576 1.58561067 1.44474939
[58] 1.31640184 1.19945633 1.09289993
[61] 0.99580971 0.90734472 0.82673872
[64] 0.75329353 0.68637301 0.62539752
[67] 0.56983893 0.51921601 0.47309029
[70] 0.43106225 0.39276787 0.35787546
[73] 0.32608279 0.29711449 0.27071966
[76] 0.24666967 0.22475622 0.20478949
[79] 0.18659656 0.17001984 0.15491574
[82] 0.14115345 0.12861377 0.11718808
[85] 0.10677741 0.09729160 0.08864848
[88] 0.08077320 0.07359753 0.06705933
[91] 0.06110196 0.05567383 0.05072792
[94] 0.04622139 0.04211521 0.03837381
[97] 0.03496479 0.03185861 0.02902838
[100] 0.02644958
```

[Hide](#)

```
predictlasso1 <- predict(modellasso,newx=X[test,],s=100)
mean((predictlasso1-y[test])^2)
```

```
[1] 151898.1
```

[Hide](#)

```
predictlasso2 <- predict(modellasso,newx=X[test,],s=50)
mean((predictlasso2-y[test])^2)
```

```
[1] 144989
```

[Hide](#)

```
predictlasso3 <- predict(modellasso,newx=X[test,],s=200)
mean((predictlasso3-y[test])^2)
```

```
[1] 183137.7
```

Hide

```
predictlasso1a <- predict(modellasso,newx=X[test,],s=100,exact=T,x=X[train,],y=y[train])
mean((predictlasso1a-y[test])^2)
```

```
[1] 151899.3
```

Hide

```
predictlasso2a <- predict(modellasso,newx=X[test,],s=50,exact=T,x=X[train,],y=y[train])
mean((predictlasso2a-y[test])^2)
```

```
[1] 145007
```

Hide

```
predictlasso3a <- predict(modellasso,newx=X[test,],s=200,exact=T,x=X[train,],y=y[train])
mean((predictlasso3a-y[test])^2)
```

```
[1] 183127.9
```

Hide

```
predictlasso4 <- predict(modellasso,newx=X[test,],s=0,exact=T,x=X[train,],y=y[train])
mean((predictlasso4-y[test])^2)
```

```
[1] 167213.2
```

Hide

```
predictlasso5 <- predict(modellasso,newx=X[test,],s=10^10,exact=T,x=X[train,],y=y[train])
mean((predictlasso5-y[test])^2)
```

```
[1] 224669.9
```

Cross-validation: By default, you perform 10 fold cross validation. Note that glmnet uses randomization in choosing the folds which we should be able to control better by setting the seed to be the same. The optimal value of lambda found from cross validation is 8.461927. You can plot the relation between the lambda parameter and the cross-validated mean error (cvm). We see that the best fit from model with the optimal lambda gives a much smaller error on the test set (143803.3) than the model which is based on complete linear regression or the model with only an intercept. We can print out the coefficients to identify that 10 predictor variables are chosen (excluding the intercept).

Hide

```
set.seed(2)
cvlasso <- cv.glmnet(X[train,],y[train])
cvlasso$glmnet.fit
```


Call: glmnet(x = X[train,], y = y[train])

	Df	%Dev	Lambda
1	0	0.00	264.500
2	1	6.57	241.000
3	1	12.03	219.600
4	2	16.64	200.100
5	2	20.51	182.300
6	3	24.00	166.100
7	3	27.88	151.400
8	3	31.11	137.900
9	3	33.78	125.700
10	3	36.00	114.500
11	3	37.85	104.300
12	3	39.38	95.050
13	4	40.72	86.610
14	4	42.28	78.920
15	6	43.60	71.910
16	6	44.73	65.520
17	6	45.67	59.700
18	7	46.55	54.390
19	8	47.33	49.560
20	8	48.00	45.160
21	8	48.55	41.150
22	10	49.02	37.490
23	8	49.43	34.160
24	8	49.78	31.130
25	9	50.14	28.360
26	9	50.47	25.840
27	10	50.81	23.550
28	10	51.10	21.450
29	10	51.35	19.550
30	10	51.55	17.810
31	10	51.72	16.230
32	11	51.95	14.790
33	11	52.36	13.470
34	11	52.70	12.280
35	11	52.99	11.190
36	10	53.21	10.190
37	10	53.38	9.287
38	10	53.52	8.462
39	10	53.63	7.710
40	11	53.74	7.025
41	11	53.87	6.401
42	12	54.09	5.832
43	12	54.28	5.314
44	12	54.44	4.842
45	12	54.57	4.412
46	13	54.73	4.020
47	13	54.91	3.663
48	13	55.06	3.338
49	13	55.19	3.041
50	13	55.29	2.771
51	14	55.44	2.525

52	14	55.87	2.300
53	14	56.25	2.096
54	15	56.57	1.910
55	15	56.86	1.740
56	15	57.10	1.586
57	15	57.30	1.445
58	15	57.47	1.316
59	16	57.61	1.199
60	17	57.73	1.093
61	17	57.85	0.996
62	17	57.96	0.907
63	17	58.05	0.827
64	18	58.14	0.753
65	17	58.20	0.686
66	17	58.25	0.625
67	16	58.29	0.570
68	16	58.32	0.519
69	16	58.34	0.473
70	16	58.36	0.431
71	16	58.38	0.393
72	17	58.40	0.358
73	17	58.41	0.326
74	17	58.42	0.297
75	19	58.44	0.271
76	19	58.45	0.247
77	19	58.49	0.225
78	19	58.49	0.205
79	19	58.52	0.187
80	19	58.53	0.170
81	19	58.54	0.155
82	19	58.55	0.141
83	19	58.56	0.129
84	19	58.57	0.117
85	19	58.58	0.107
86	19	58.58	0.097
87	19	58.59	0.089
88	19	58.59	0.081
89	19	58.59	0.074
90	19	58.60	0.067
91	19	58.60	0.061
92	19	58.60	0.056
93	19	58.60	0.051
94	19	58.60	0.046
95	19	58.61	0.042
96	19	58.61	0.038
97	19	58.61	0.035
98	19	58.61	0.032
99	19	58.61	0.029
100	19	58.61	0.026

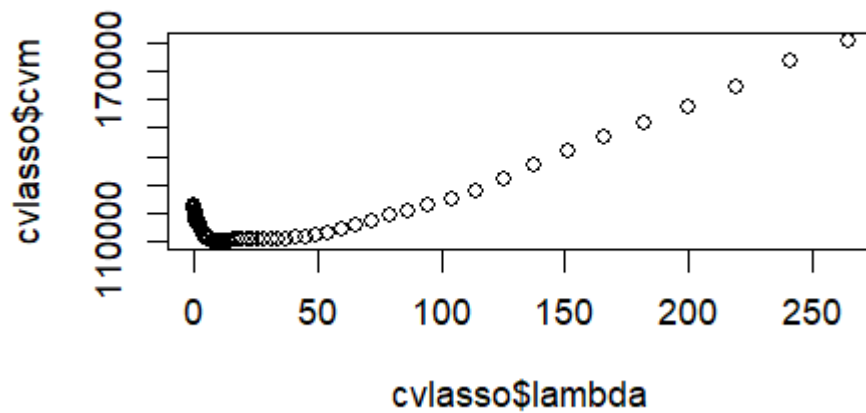
[Hide](#)

cvlasso\$lambda.min

[1] 8.461927

[Hide](#)

```
plot(cvlasso$lambda,cvlasso$cvm)
```

[Hide](#)

```
predictlassocv <- predict(modellasso,s=8.461927,newx=X[test,])  
mean((predictlassocv-y[test])^2)
```

```
[1] 143807.7
```

[Hide](#)

```
coef(modellasso,s=8.461927)
```

20 x 1 sparse Matrix of class "dgCMatrix"

```
      s=8.461927  
(Intercept) 121.9078071  
AtBat        .  
Hits         .  
HmRun        3.4527540  
Runs         .  
RBI          .  
Walks        4.0095547  
Years        -13.2996318  
CAtBat       .  
CHits        0.2172280  
CHmRun       0.8938979  
CRuns        .  
CRBI         0.3905921  
CWalks       .  
LeagueN     49.0732189  
DivisionW    -136.8585423  
PutOuts      0.1182314  
Assists      0.1424964  
Errors       .  
NewLeagueN   .
```

Hide

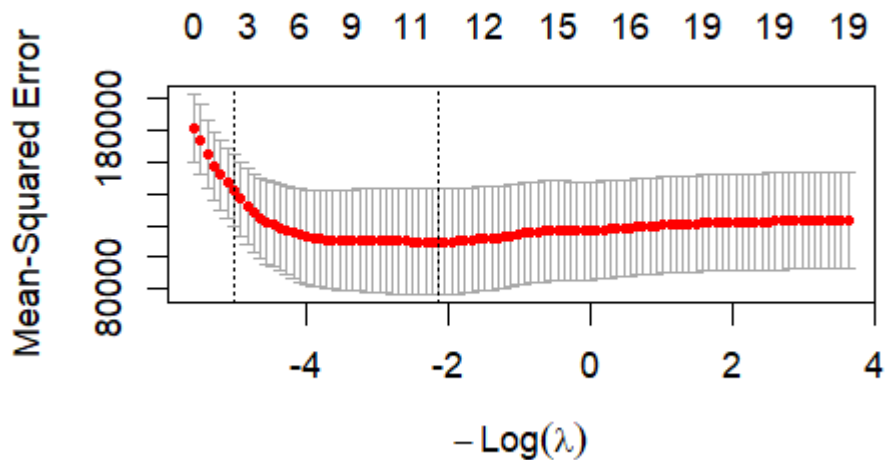
```
predictlassocvexact <- predict(modellasso,s=8.461927,newx=X[test,],exact=T,x=X[train,],y=y[train])
mean((predictlassocvexact-y[test])^2)
```

```
[1] 143803.3
```

You can also plot the output (cross-valuation error curve) directly for LASSO as follows:

Hide

```
plot(cvlasso)
```



Here's what each element means:

X-axis (bottom): $-\log(\lambda)$ — as you move left, λ increases (more regularization, fewer variables). Moving right means less regularization.

X-axis (top): The number of non-zero coefficients in the model at each λ value.

Y-axis: Cross-validated mean-squared error (CV MSE), averaged across the 10 folds.

Red dots: The CV MSE at each λ tried during fitting.

Gray bars: ± 1 standard error around each CV MSE estimate, reflecting variability across folds.

Left dashed line (lambda.min): The λ that minimizes CV MSE — this is the best-fitting model ($\lambda \approx 8.46$, 10 predictors in our case).

Right dashed line (lambda.1se): The largest λ that those CV MSE is within 1 standard error of the minimum — a more parsimonious model that trades a small increase in error for fewer predictors ($\lambda \approx 151.35$, 3 predictors in our case).

The wide gray bands indicate there's meaningful variability in the CV error, so the difference between models in the flat region of the curve is not large.

In the implementation of LASSO in glmnet the model that is solved is given by:

$$\min \frac{1}{2n} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_1 - \dots - \beta_p x_p)^2 + \lambda \sum_{j=1}^p |x_j|$$

and scale (standardize) the variable before running the fit. The coefficients are returned on the original scale.

You can look at <https://glmnet.stanford.edu/articles/glmnet.html>

(<https://glmnet.stanford.edu/articles/glmnet.html>) for details.